UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Análise de escalabilidade de um algoritmo de migração reversa no tempo

Júlia Alanne Silvino dos Santos

Orientador: Prof. Dr. Samuel Xavier de Souza

Natal, RN, 6 de julho de 2026

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Análise de escalabilidade de um algoritmo de migração reversa no tempo

Júlia Alanne Silvino dos Santos

Orientador: Prof. Dr. Samuel Xavier de Souza

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 6 de julho de 2026

Divisão de Serviços Técnicos

Catalogação da publicação na fonte. UFRN / Biblioteca Central Zila Mamede

Santos, Júlia Alanne Silvino dos
Análise de escalabilidade de um algoritmo de migração reversa no tempo / Júlia Alanne Silvino dos Santos - 6 de julho de 2026
50 p.

Orientador: Prof. Dr. Samuel Xavier de Souza

Monografia - Universidade Federal do Rio Grande do Norte. Centro de Tecnologia. Graduação em Engenharia de Computação.

1.Análise de Escalabilidade - Monografia. 2. Migração reverssa no tempo (RTM) - Monografia. 3. Eficiência - Monografia. 4. Speedup - Monografia. I. Souza, Samuel Xavier de. II. Fernandes, João B. III. Título.

RN/UF/BCZM CDU XXX.YYY(YYY.0)

Análise de escalabilidade de um algoritmo de migração reversa no tempo

Júlia Alanne Silvino dos Santos

Monografia aprovada em 30 de Junho de 2026, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Samuel Xavier de Souza (orientador) . . . . . DCA/UFRN

Me. João Batista Fernandes . . . . . DCA/UFRN

Prof. Dr. Kayo Gonçalves e Silva . . . . . IMD/UFRN

Agradecimentos

A Deus, por estar sempre presente em minha vida.

À minha família pelo apoio durante esta jornada.

Ao meu orientador Samuel Xavier de Souza pela dedicação na orientação neste trabalho.

A João Batista Fernandes e ao professor Kayo Gonçalves e Silva, pela colaboração no desenvolvimento deste trabalho.

Ao Núcleo de Processamento de Alto Desempenho (NPAD/UFRN), pelo suporte computacional.

Resumo

A migração reversa no tempo (RTM) é um método de alta precisão utilizado na indústria de petróleo e gás para a identificação de reservatórios. Contudo, esse método impõe um alto custo computacional e de memória. Dessa forma, é fundamental que algoritmos de RTM sejam escaláveis, de modo a fazer uso eficiente dos recursos computacionais disponíveis. Nesse contexto, a ferramenta PaScal Suite foi desenvolvida para auxiliar na análise de escalabilidade de aplicações paralelas. Com isso, o presente trabalho apresenta uma análise de escalabilidade de um algoritmo de RTM utilizando a suíte de ferramentas PaScal. A análise possibilitou mapear as regiões que ditam o comportamento da escalabilidade da aplicação, bem como identificar gargalos críticos de concorrência e sincronização. Os resultados indicaram que o programa mantém uma boa eficiência em cenários de paralelismo moderado, mas sofre degradação com o aumento massivo de *threads*. Diante disso, este estudo estabelece uma base para direcionar futuras otimizações no código-fonte, visando mitigar as limitações de desempenho identificadas.

Palavras-chave: Análise de Escalabilidade, RTM, Eficiência, *Speedup*, PaScal Suite.

Abstract

Reverse Time Migration (RTM) is a high-precision method used in the oil and gas industry for reservoir identification. However, this method imposes high computational and memory demands. Thus, it is fundamental that RTM algorithms are scalable to make efficient use of available computational resources. In this context, the PaScal Suite was developed to assist in the scalability analysis of parallel applications. Therefore, this work presents a scalability analysis of an RTM algorithm using the PaScal suite. The analysis enabled the mapping of the regions that dictate the application's scalability behavior, as well as the identification of critical concurrency and synchronization bottlenecks. Results indicated that the program maintains good efficiency in moderate parallelism scenarios, but suffers degradation with a massive increase in *threads*. Therefore, this study establishes a foundation to guide future source code optimizations, aiming to mitigate the identified performance limitations.

Keywords: Scalability Analysis, RTM, Efficiency, Speedup, PaScal Suite.

Sumário

Sumário i

Lista de Figuras iii

Lista de Tabelas v

1 Introdução 1

2 Computação de Alto Desempenho 3
2.1 Threads e Processos 4
2.2 Métricas de Desempenho Paralelo 4
2.2.1 Speedup 5
2.2.2 Lei de Amdahl 5
2.2.3 Eficiência 5
2.3 Escalabilidade 6

3 Sísmica de Reflexão 7
3.1 Migração Reversa no Tempo 8
3.1.1 Propagação e Retropropagação de Campos de Ondas 9
3.1.2 Condição de Imagem 11
3.2 Algoritmo RTM-3D 12
3.2.1 Granularidade 12

4 PaScal Suite 15
4.1 PaScal Analyzer 15
4.2 PaScal Viewer 17

5 Experimentos 19

6 Resultados 23

7 Conclusão 29

Referências bibliográficas 30

Lista de Figuras

3.1 Processo de aquisição sísmica no oceano. 7
3.2 Diagrama do algoritmo de migração reversa no tempo. 9
3.3 Exemplo de modelo de velocidades 3D. 10

4.1 Fluxo de trabalho do PaScal Analyzer. 16
4.2 Diagrama de eficiência gerado pelo PaScal Viewer. 18

5.1 Representação bidimensional do modelo de velocidades com a delimitação dos recortes experimentais (i_1, i_2, i_3, i_4). 21

6.1 Diagrama de eficiência referente à execução completa da aplicação. . . . 23
6.2 Diagrama de eficiência comparativo das etapas de Propagação direta e Retropropagação. 24
6.3 Diagrama de eficiência da Condição de Imagem. 24
6.4 Diagrama de eficiência comparativo do cálculo de propagação direta e reversa. 25
6.5 Diagrama de eficiência comparativo das compensações por iluminação. . 26
6.6 Diagrama de eficiência comparativo das regiões responsáveis por salvar e recuperar os *checkpoints*. 27

Lista de Tabelas

5.1 Dimensões das malhas dos modelos de velocidades utilizados nos experimentos. 20

Capítulo 1

Introdução

A Migração Reversa no Tempo (RTM, do inglês: *Reverse Time Migration*) segundo Zhou et al. (2018), é um método avançado de imageamento sísmico utilizado para mapear a subsuperfície em regiões de alta complexidade geológica. O método baseia-se na solução numérica da equação da onda completa, permitindo simular a propagação dos campos de onda. O algoritmo da RTM é composto por três etapas principais, sendo estas a modelagem direta, que consiste na propagação no tempo do campo de ondas gerado pela fonte sísmica; a retropropagação no tempo, onde os dados registrados na superfície (ondas refletidas) são injetados de volta no modelo geológico; e a aplicação de uma condição de imagem, que combina ambos os campos de onda em cada ponto do domínio espacial para gerar a imagem final.

Para o cálculo da condição de imagem, conforme Zhou et al. (2018), o algoritmo exige o acesso ao campo de ondas da fonte (propagação) de maneira retroativa no tempo enquanto calcula simultaneamente o campo adjunto (retropropagação). Assim, torna-se necessária a resolução da equação da onda para milhares de passos de tempo em malhas computacionais densas. Dessa forma, a dimensionalidade dos dados impõe um custo computacional e de armazenamento à RTM. Aplicações com alto custo computacional, como a RTM, precisam ser escaláveis, utilizando de forma eficiente os recursos de Computação de Alto Desempenho (HPC, do inglês: *High Performance Computing*).

Neste contexto, realizar a análise da escalabilidade em algoritmos RTM é fundamental não apenas para identificar gargalos, mas também para avaliar o comportamento do código com o aumento de recursos computacionais, como o número de núcleos de processamento. Essa avaliação permite mensurar métricas fundamentais, tais como o speedup e a eficiência paralela, garantindo que o algoritmo aproveite adequadamente a capacidade das arquiteturas modernas de processamento.

O conjunto de ferramentas PaScal Suite auxilia no processo de análise de escalabilidade, sendo composto pelas ferramentas PaScal Analyzer e PaScal Viewer, projetadas para automatizar a coleta de dados a partir de múltiplas execuções e facilitar a visualização das métricas de desempenho, respectivamente.

O presente trabalho tem como objetivo realizar a análise de escalabilidade de um algoritmo de RTM implementado em C++, fazendo uso das ferramentas do PaScal Suite, para investigar o desempenho do algoritmo sob diferentes configurações de paralelização no ambiente de supercomputação do Núcleo de Processamento de Alto Desempenho

(NPAD)[1].

O restante deste documento está organizado da seguinte forma: o Capítulo 2 aborda conceitos fundamentais para computação de alto desempenho como threads e processos, além de algumas métricas de desempenho de HPC, como speedup, eficiência e escalabilidade. O Capítulo 3 apresenta a fundamentação teórica acerca do algoritmo RTM, destacando algumas contribuições relevantes da literatura. O Capítulo 4 detalha o PaScal Suite, apresentando suas características e funcionalidades. O Capítulo 5 descreve o setup dos experimentos, detalhando a arquitetura do nó utilizado, as configurações de entrada e os dados sintéticos aplicados aos testes. O Capítulo 6 expõe e discute os resultados obtidos, utilizando as visualizações geradas pelo PaScal Viewer para analisar a eficiência global da aplicação e de regiões específicas do código. Por fim, o Capítulo 7 apresenta as considerações finais sobre os resultados de escalabilidade, além de apresentar perspectivas para trabalhos futuros.

[1] https://npad.ufrn.br/npad/npad

Capítulo 2

Computação de Alto Desempenho

A Lei de Moore previu que o número de transistores em circuitos integrados dobraria a cada dois anos. No entanto, o aumento da densidade de transistores e da frequência de *clock* atingiu uma barreira física, em especial devido ao alto consumo de energia e à dissipação térmica. Com essa limitação, a indústria migrou do aumento da performance de núcleo único (*single-core*) para arquiteturas multinúcleo (*multi-core*).

A era *multi-core* trouxe consigo uma implicação: o aumento do número de unidades de processamento, apesar de elevar o poder computacional, não melhora automaticamente o desempenho de algoritmos sequenciais. Isso ocorre porque estes são projetados para um único fluxo de instruções, de modo que não levam em consideração a existência de múltiplos processadores.

Conforme Pacheco (2011), embora seja possível executar várias instâncias independentes do mesmo algoritmo sequencial em vários processadores disponíveis, a grande maioria das aplicações complexas não se beneficia significativamente dessa abordagem.

Dessa forma, uma alternativa seria realizar a autoparalelização do código, utilizando compiladores que convertem programas sequenciais em programas paralelos de forma automatizada. Essas ferramentas reconhecem estruturas de repetição comuns e as traduzem em estruturas concorrentes. Contudo, sem a supervisão direta de um desenvolvedor, essa técnica tende a produzir código ineficiente.

Nesse cenário, o desenvolvimento de algoritmos paralelos tornou-se fundamental para extrair o potencial máximo das arquiteturas multinúcleo, permitindo que grandes volumes de dados sejam processados simultaneamente. Entretanto, algumas aplicações, ou seções específicas de um código, funcionam melhor de forma sequencial, de modo que paralelizar essas tarefas não aumenta o desempenho e pode, inclusive, degradá-lo. Isso ocorre devido ao *overhead* paralelo, que consiste no custo computacional extra associado à criação de *threads*, à sincronização, à comunicação e ao escalonamento de tarefas.

As arquiteturas paralelas podem ser divididas de acordo com a organização de sua memória (Diaz et al. 2012), sendo classificadas em compartilhadas e distribuídas. As arquiteturas de memória compartilhada possuem um único espaço de endereçamento acessível a todos os processadores. Em contrapartida, nas arquiteturas de memória distribuída, cada processador possui seu próprio espaço de endereçamento.

Em sistemas de memória compartilhada, geralmente adota-se o modelo de programação da API *OpenMP* (do inglês: *Open Multi-Processing*)(OpenMP Architecture Review

Board 2015). Já para sistemas de memória distribuída, utiliza-se comumente o modelo de programação por troca de mensagens, sendo a API *MPI* (*Message Passing Interface*) a mais adotada.

Nesse contexto, este capítulo aborda conceitos fundamentais da computação de alto desempenho, como *threads* e processos, além de detalhar as métricas de avaliação de desempenho, tais como *speedup*, eficiência e escalabilidade.

2.1 Threads e Processos

Processos e *threads* são conceitos fundamentais em computação paralela. Um processo pode ser definido como uma instância de um programa em execução, para o qual o sistema operacional aloca um espaço de memória próprio e isolado. Em função desse isolamento, a comunicação entre processos distintos geralmente precisa ser realizada por meio de troca de mensagens.

Por outro lado, uma *thread* é a menor unidade de processamento escalonável pelo sistema operacional. As *threads* de um processo compartilham o mesmo espaço de endereçamento e os recursos alocados a esse processo. Essa característica permite que múltiplas *threads* de um mesmo processo se comuniquem rapidamente. Elas permitem dividir um programa em tarefas relativamente independentes que podem ser executadas concorrentemente. Além disso, possuem a propriedade de que, mesmo que uma ou mais *threads* sejam bloqueadas pelo sistema operacional, as demais *threads* podem continuar sendo executadas normalmente.

A criação e a destruição de processos são mais custosas do que as de *threads* devido à complexidade da sua estrutura. Com isso, os processos normalmente tendem a ter um ciclo de vida relativamente longo, enquanto as *threads* são criadas e destruídas dinamicamente conforme a necessidade de computação (Pacheco 2011).

2.2 Métricas de Desempenho Paralelo

Em um cenário ideal, a execução de um código que demanda um tempo T de forma sequencial levaria um tempo correspondente a T/p ao ser paralelizado, sendo p o número de núcleos de processamento utilizados. Contudo, na prática, diversos fatores contribuem para a perda de desempenho em aplicações paralelas. Entre os principais, destacam-se os *overheads* de sincronização e gerenciamento das *threads*, a existência de frações do algoritmo que são estritamente sequenciais e o desbalanceamento de carga, que resulta em processadores ociosos durante a execução.

Assim, faz-se necessária uma análise acerca do desempenho dessas aplicações. Nesse contexto, as métricas mais utilizadas para a avaliação de algoritmos paralelos são o *speedup* e a eficiência, que relacionam o tempo de execução do programa paralelo com o da sua versão sequencial original.

2.2.1 Speedup

O *speedup* mede a taxa de redução do tempo de execução de um algoritmo paralelo em relação à sua execução sequencial. O cálculo do *speedup* alcançado ao utilizar p processadores (ou *threads*), denotado por S_p, é dado pela fórmula:

$$S_p = \frac{T_s}{T_p} \tag{2.1}$$

Onde T_s é o tempo de execução do algoritmo sequencial e T_p é o tempo de execução do programa paralelo utilizando p núcleos. No cenário ideal, espera-se que um programa executado com p processadores obtenha um tempo de execução p vezes mais rápido que sua versão sequencial, ou seja, alcance $S_p = p$. Esse comportamento teórico é comumente chamado de *speedup* linear ou ideal.

2.2.2 Lei de Amdahl

Na prática, o *speedup* linear é extremamente difícil de ser alcançado. De acordo com Amdahl (1967), o *speedup* máximo de um programa é limitado pela fração sequencial do código, ou seja, pelas partes do algoritmo que não podem ser paralelizadas. Se f representa a fração do código executada sequencialmente, o ganho de desempenho máximo obedece à relação:

$$S_p \leq \frac{1}{f + \frac{1-f}{p}} \tag{2.2}$$

Isso implica que, mesmo que o número de processadores seja muito grande ($p \to \infty$), o *speedup* máximo do programa ficará limitado a $1/f$. Por exemplo, em um algoritmo com a parte sequencial $f = 0,5$, seu *speedup* será $S_p \leq 2$. Dessa forma, a Lei de Amdahl mostra que a proporção de código que precisa ser executada sequencialmente determina o limite da melhoria de desempenho do algoritmo ao utilizar paralelismo.

2.2.3 Eficiência

A eficiência paralela mede a capacidade de um algoritmo de utilizar de forma otimizada os processadores. Ou seja, ela atua como uma métrica normalizada do *speedup*, indicando o quanto cada núcleo de processamento é, de fato, utilizado:

$$E_p = \frac{S_p}{p} \tag{2.3}$$

Onde E_p é a eficiência, S_p é o *speedup* alcançado e p é o número de núcleos de processamento.

Em um cenário de *speedup* ideal (ou linear), onde o ganho de tempo acompanha perfeitamente a adição de núcleos ($S_p = p$), a eficiência atinge o valor máximo de 1 (ou 100%). Isso indica que foi utilizada a máxima capacidade de todos os processadores. Na prática, devido às perdas de desempenho inerentes à computação paralela, como o

overhead de sincronização e o tempo de acesso à memória, a eficiência costuma ser menor que 1 e tende a decair progressivamente com o aumento do número de processadores.

Contudo, em casos excepcionais, o algoritmo pode apresentar um *speedup* superlinear. Nessas situações, o valor da eficiência torna-se maior que 1, fenômeno que geralmente ocorre devido a efeitos favoráveis na hierarquia de memória, como o melhor aproveitamento do *cache* local dos processadores quando os dados do problema são particionados em pedaços menores.

2.3 Escalabilidade

Segundo Pacheco (2011), um programa é considerado escalável se for possível encontrar uma taxa de aumento correspondente no tamanho do problema de modo que a aplicação mantenha uma eficiência E. Ou seja, a escalabilidade indica a capacidade de um sistema paralelo de manter ou até melhorar seu desempenho à medida que o número de núcleos de processamento ou a complexidade do problema aumenta. Na computação de alto desempenho, essa propriedade é avaliada sob duas perspectivas principais:

- **Escalabilidade Forte (*Strong Scaling*):** Mede o desempenho quando o tamanho do problema é mantido fixo, enquanto os recursos computacionais aumentam. O objetivo principal é reduzir o tempo computacional ao aumentar os recursos. Dessa forma, um programa apresenta escalabilidade forte se o seu tempo de execução diminui proporcionalmente ao número de núcleos adicionados. Assim, para um algoritmo que processa um volume de dados de tamanho fixo, no cenário ideal, se ele for fortemente escalável, o seu tempo de execução cairá pela metade com a duplicação do número de processadores. Entretanto, alcançar escalabilidade forte costuma ser um desafio, tendo em vista que, ao aumentar a quantidade de processadores para um problema fixo, as *threads* tendem a ficar ociosas, e os custos fixos de sincronização e gerenciamento passam a dominar o tempo total de execução.
- **Escalabilidade Fraca (*Weak Scaling*):** Mede o desempenho quando o tamanho do problema cresce de forma proporcional ao aumento dos recursos computacionais. Assim, um programa é considerado fracamente escalável quando consegue manter a eficiência constante à medida que os recursos e a dimensão do problema aumentam. Por exemplo, se o volume de dados a ser processado por um algoritmo for duplicado e os recursos computacionais forem dobrados simultaneamente, ele será fracamente escalável se conseguir manter a sua eficiência.

Capítulo 3

Sísmica de Reflexão

A sísmica de reflexão é um método amplamente utilizado para o estudo do subsolo. A indústria de exploração de hidrocarbonetos faz amplo uso dessa técnica devido à grande profundidade em que os materiais de interesse se encontram, sendo fundamental para a identificação de reservatórios de petróleo e gás. Na busca por reservatórios, conforme Yilmaz (2001), a exploração sísmica divide-se em três etapas principais: aquisição, processamento e interpretação dos dados.

A etapa de aquisição, ilustrada para o ambiente marinho na Figura 3.1, inicia-se com o acionamento de uma fonte sísmica, que no oceano geralmente consiste em canhões de ar rebocados por um navio, enquanto em terra utilizam-se caminhões vibratórios ou cargas explosivas. No contexto da exploração, cada acionamento dessa fonte é conhecido como tiro (*shot*). Conforme esquematizado na figura, as ondas se propagam em direção ao subsolo, viajando pelo meio até encontrarem as interfaces de reflexão, que são os limites entre camadas geológicas com propriedades físicas distintas. Nessas interfaces, parte da energia da onda incidente é refletida de volta à superfície e outra parte é transmitida. Ao retornar à superfície, a onda é registrada por um conjunto de receptores, chamados de geofones para exploração em terra, ou hidrofones para exploração no mar (também ilustrados na figura). A informação capturada por cada receptor ao longo do tempo é denominada traço sísmico, e o conjunto desses traços compõe o sismograma, o qual é comumente armazenado em formato binário.

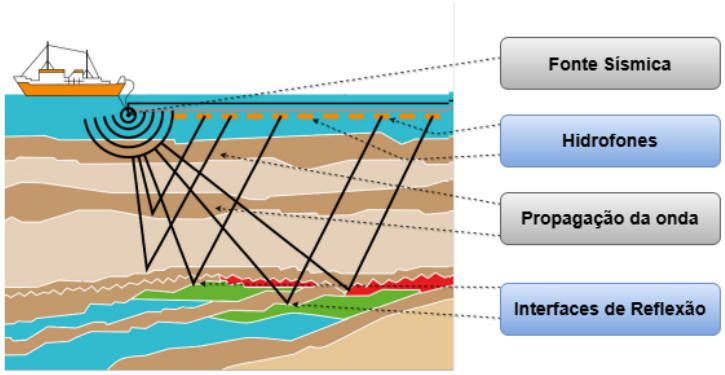


Figura 3.1: Processo de aquisição sísmica no oceano. Fonte: Adaptado de Fernandes (2020).

Após a etapa de aquisição, é realizado o processamento. Nessa fase, os dados registrados são manipulados para gerar uma imagem estrutural da subsuperfície, representando uma seção sísmica com varios traços representados no tempo e na profundidade. Como os dados brutos sofrem influência de condições irregulares de aquisição, como ruídos e difrações, aplicam-se técnicas de pré-processamento (como o uso de filtros e ganhos de amplitude) para mitigar esses artefatos. O objetivo central do processamento de dados de reflexão é aumentar a relação Sinal-Ruído (SNR, do inglês *Signal-to-Noise Ratio*) e melhorar a resolução dos traços sísmicos.

Segundo Yilmaz (2001), o processamento pode ser dividido em três etapas essenciais: deconvolução, empilhamento e migração. A deconvolução visa melhorar a resolução temporal, comprimindo o efeito da assinatura da fonte contido nos traços sísmicos. O empilhamento consiste na soma de traços redundantes para aumentar a relação sinal-ruído e reduzir significativamente o volume de dados a serem processados. Por fim, a migração reposiciona as interfaces de reflexão para as suas verdadeiras posições espaciais no subsolo e reduz os efeitos de difração, gerando, assim, a imagem sísmica com alta resolução.

A interpretação constitui a última etapa da exploração sísmica. Nessa fase, é selecionada uma superfície de tempo de reflexão associada a uma camada limite a partir de um volume de dados migrado no tempo, ou interfaces refletoras em um volume de dados migrado em profundidade. Tradicionalmente, na interpretação de dados sísmicos, é escolhida uma superfície ou uma seção migrada em 2D. Entretanto, as opções de visualização em 3D oferecem mais informações sobre a região de interesse.

3.1 Migração Reversa no Tempo

Conforme detalhado anteriormente, a migração constitui a etapa final do processamento para exploração sísmica. Nesta fase, ocorre o reposicionamento das interfaces de reflexão para as suas verdadeiras posições espaciais, resultando na geração de uma imagem estrutural de alta resolução da subsuperfície. Enquanto a deconvolução e o empilhamento atuam na preparação e redução do volume de informações, a migração representa o maior gargalo de custo computacional e de memória de todo o fluxo de exploração.

Entre as técnicas de migração utilizadas para o mapeamento de estruturas geológicas na subsuperfície, a Migração Reversa no Tempo se destaca por gerar imagens de alta resolução, auxiliando na identificação precisa dessas formações. Assim, esse método tornou-se uma ferramenta fundamental na indústria de exploração de petróleo e gás.

A base inicial para a RTM foi o desenvolvimento de um método de migração de dados sísmicos baseado na solução numérica de equações diferenciais parciais, desenvolvido por Claerbout (1971) e Claerbout & Doherty (1972). Fundamentados nesses conceitos, Kosloff & Baysal (1983) e Baysal et al. (1983) propuseram uma abordagem alternativa para a migração por diferenças finitas. Estes utilizaram a equação completa da onda acústica para realizar a extrapolação reversa no tempo, apresentando melhorias significativas na qualidade da imagem em relação aos métodos de migração existentes na época. Além da extrapolação do campo de ondas, Claerbout (1971) também introduziu o princípio de imageamento clássico. Esse princípio serviu de base para as diversas condições de imagem utilizadas no RTM. Uma análise detalhada das principais condições de imagem e

seus impactos na qualidade da migração pode ser encontrada no trabalho de Silva (2009).

De acordo com Lu & Magerlein (2013), a RTM se baseia na intuição física de que as ondas de pressão se correlacionam nas interfaces refletoras. Assim, a RTM realiza a correlação entre os campos de ondas incidentes e refletidas com o objetivo de mapear espacialmente essas interfaces.

O algoritmo da RTM pode ser dividido em três etapas principais. A primeira é a propagação do campo de ondas direto, gerado pela fonte sísmica, que consiste em uma perturbação (como um pulso de ar comprimido ou um explosivo) introduzida na subsuperfície. A segunda etapa é a retropropagação, na qual os sismogramas com os dados registrados na superfície são injetados de volta no modelo computacional a partir das posições dos receptores. Por fim, ocorre a aplicação da condição de imagem, que realiza a correlação temporal entre os campos de onda propagado e retropropagado em cada ponto da malha para construir a imagem final. Repetindo esse processo para todos os tiros do dado sísmico. A Figura 3.2 apresenta um diagrama do funcionamento básico da RTM.

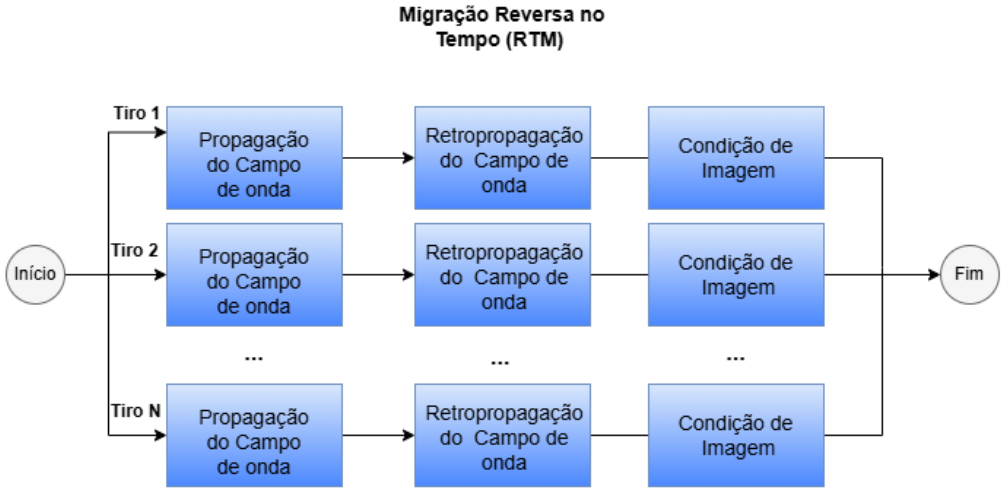


Figura 3.2: Diagrama do algoritmo de migração reversa no tempo.

3.1.1 Propagação e Retropropagação de Campos de Ondas

As etapas de propagação de campos de ondas na RTM utilizam, predominantemente, algoritmos baseados no Método das Diferenças Finitas (FDM, do inglês: *Finite Difference Method*). Nesse método, o domínio físico do problema é discretizado, formando uma malha tridimensional ou bidimensional. O FDM tem como objetivo aproximar as derivadas parciais presentes na equação diferencial por meio de uma expansão truncada da série de Taylor. Assim, a versão discreta da equação da onda é resolvida iterativamente com o avanço no tempo, aplicando condições iniciais e condições de contorno numéricas, explicadas na Seção 3.1.1, com o objetivo de delimitar o problema.

Segundo Lara (2012), a modelagem elástica descreve de forma mais completa a propagação da onda sísmica, tendo em vista que leva em consideração tanto as ondas compressivas (ondas P) quanto as ondas cisalhantes (ondas S) no meio geológico. As ondas S

são responsáveis pelo movimento de partículas do meio na direção perpendicular à direção de propagação da onda, enquanto as ondas P fazem com que as partículas oscilem na mesma direção da propagação. Entretanto, a modelagem acústica, embora considere apenas a propagação de ondas P, fornece resultados bastante satisfatórios no processamento sísmico e apresenta um custo computacional significativamente menor.

Após a simulação da fonte sísmica e a geração da onda acústica, a propagação pode ser modelada através da solução da equação da onda 3D. Para um meio acústico com densidade constante, a equação da onda é dada por:

$$\frac{\partial^2 u(\mathbf{x},t)}{\partial x_1^2} + \frac{\partial^2 u(\mathbf{x},t)}{\partial x_2^2} + \frac{\partial^2 u(\mathbf{x},t)}{\partial x_3^2} = \frac{1}{c(\mathbf{x})^2}\frac{\partial^2 u(\mathbf{x},t)}{\partial t^2} + s(\mathbf{x},t) \tag{3.1}$$

Em que $\mathbf{x} = (x_1, x_2, x_3)$ representam as coordenadas espaciais tridimensionais, $u(\mathbf{x},t)$ é o campo de pressão acústica, $c(\mathbf{x})$ é o modelo de velocidade acústica e $s(\mathbf{x},t)$ é a função da fonte sísmica no tempo t.

O modelo de velocidade, representado por $c(\mathbf{x})$, é utilizado em ambas as propagações. Ele especifica a velocidade de propagação da onda em cada ponto. Na Figura 3.3, é apresentado um exemplo de modelo de velocidade 3D composto por duas camadas de velocidades distintas, sendo estas c_1 e c_2.

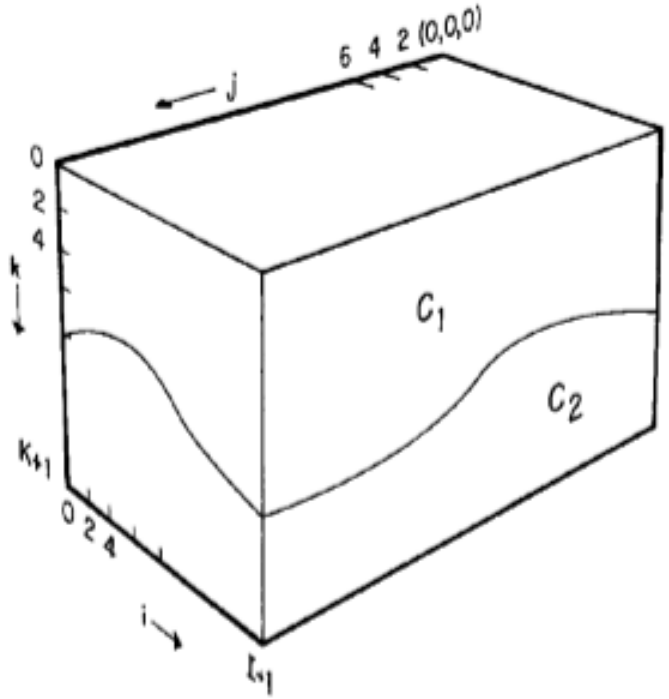


Figura 3.3: Exemplo de modelo de velocidades 3D. Fonte: Mufti (1990).

Assim, a fonte gera as ondas que são propagadas para a frente através do modelo de velocidade, gerando os campos de onda incidentes. Em seguida, a informação registrada

no sismograma é propagada para trás através do mesmo modelo, gerando o campo de onda refletido (retropropagação). Nesse caso, a própria informação do sismograma atua como a fonte. Por fim, essas duas propagações são correlacionadas, gerando a imagem.

Restrições Numéricas

Na propagação de ondas, para que a resolução da equação por diferenças finitas seja consistente, duas restrições devem ser observadas: uma restrição espacial e uma restrição temporal. Ignorar essas restrições pode introduzir erros de dispersão e instabilidade numérica. A restrição espacial, dada pela Equação 3.2, limita o espaçamento máximo entre os pontos da malha do modelo. Essa condição impede a dispersão de energia.

$$\max(\Delta x, \Delta y, \Delta z) \leq \frac{c_{min}}{\omega \cdot f_{max}} \tag{3.2}$$

Em que Δx, Δy e Δz são os intervalos de amostragem espacial nas direções x, y e z. c_{min} é a menor velocidade do modelo, f_{max} é a frequência máxima da fonte $s(t)$ e ω é o número de amostras por comprimento de onda mínimo. Para esquemas de diferenças finitas de alta ordem, segundo Carcione et al. (2002), ω deve ser maior ou igual a 4.

A restrição temporal, dada pela Equação 3.3, estabelece o critério de estabilidade da simulação.

$$\Delta t \leq \frac{2\min(\Delta x, \Delta y, \Delta z)}{c_{max}\pi\sqrt{3}} \tag{3.3}$$

Em que Δt é a amostragem no tempo e c_{max} é o valor máximo de velocidade do modelo.

3.1.2 Condição de Imagem

A condição de imagem constitui a terceira etapa do algoritmo da RTM, tendo como objetivo correlacionar os campos de onda para gerar a imagem estrutural. Nessa fase, os campos de onda incidente (fonte) e refletido (receptor) são correlacionados ponto a ponto a cada passo de tempo para gerar a imagem final.

Dentre as diferentes formulações disponíveis na literatura para essa etapa, destacam-se a condição de imagem com tempo de excitação e a condição por correlação cruzada (Zhou et al. 2018).

A condição de imagem por correlação cruzada baseia-se no princípio de que os campos de onda da fonte e do receptor se encontram, em tempo e espaço, predominantemente sobre as interfaces refletoras. Fora dessas interfaces, o produto entre os campos tende a zero quando integrado ao longo do tempo. Essa relação é expressa por:

$$I(\mathbf{x}) = \int_0^{T_{max}} S(\mathbf{x},t) \cdot R(\mathbf{x},t)\, dt \tag{3.4}$$

Em que $I(\mathbf{x})$ representa a amplitude da imagem final na posição espacial tridimensional $\mathbf{x} = (x,y,z)$, enquanto $S(\mathbf{x},t)$ e $R(\mathbf{x},t)$ correspondem aos campos de onda da fonte e do receptor, respectivamente, integrados ao longo do tempo total de propagação T_{max}.

Iluminação

A condição de imagem tende a gerar artefatos de baixa frequência na imagem final. Esses ruídos, frequentemente observados nas porções mais rasas do modelo, ocorrem devido à correlação de ondas que se propagam na mesma direção, ondas refratadas e ondas retroespalhadas (Zhou et al. 2018).

Dessa forma, uma das estratégias para mitigar esses artefatos é aplicar a compensação pela iluminação Zhou et al. (2018). Esse método normaliza a imagem dividindo o resultado da condição de imagem pela energia acumulada do campo de onda da fonte e dos receptores. Assim, a relação entre a iluminação e a condição de imagem na RTM é fundamental para a qualidade da imagem final, agindo como um fator de escala. A condição de imagem compensada pela iluminação é expressa por:

$$I(\mathbf{x}) = \frac{\int_0^{T_{max}} S(\mathbf{x},t) \cdot R(\mathbf{x},t)\, dt}{\int_0^{T_{max}} [S^2(\mathbf{x},t) + R^2(\mathbf{x},t)]\, dt + \varepsilon} \tag{3.5}$$

Em que ε é um fator de estabilização numérico, adicionado ao denominador para evitar a divisão por zero em regiões onde a energia do campo de onda é nula.

3.2 Algoritmo RTM-3D

No presente trabalho, o objeto de estudo é a aplicação da RTM acústico 3D implementado no software Mamute, desenvolvido em C++ com o uso de técnicas de paralelização usando OpenMP. O repositório do código pode ser acessado em: `https://gitlab.com/lappsufrn/mamute`.

O Algoritmo 1 apresenta o pseudocódigo que ilustra a implementação da RTM acústica no software Mamute. Inicialmente, o algoritmo recebe o modelo de velocidade e realiza a leitura das coordenadas dos receptores e dos respectivos traços sísmicos. A partir desses dados, executa a propagação para cada passo de tempo (t_i), calculando o campo de onda direto. Em seguida, realiza a retropropagação para a obtenção do campo de onda reverso. Por fim, é aplicada a condição de imagem, seguida do cálculo da iluminação.

3.2.1 Granularidade

O algoritmo de RTM implementado no software *Mamute* é estruturado para suportar uma estratégia de paralelização em múltiplos níveis. No nível mais externo, o algoritmo permite a adoção de uma granularidade grossa focada nos tiros (*shots*), em que a distribuição sob demanda ocorre entre processos, utilizando MPI em conjunto com um escalonador *Work Stealing* (WS).

Contudo, internamente à execução de cada tiro, o código emprega uma granularidade fina usando OpenMP para paralelizar os cálculos iterativos sobre a malha de pontos do domínio espacial. Nessa abordagem, a aplicação inicia regiões paralelas (`#pragma omp parallel`) para que os trechos de maior custo computacional, como a propagação, a retropropagação do campo de onda e a condição de imagem, sejam divididos simultaneamente entre múltiplas *threads*. A distribuição da carga de trabalho espacial é delegada às

políticas de escalonamento (*schedule*) do OpenMP, garantindo o processamento eficiente das matrizes tridimensionais.

Embora o *Mamute* possua suporte para OpenMP e MPI, o escopo deste trabalho concentra-se exclusivamente na técnica de paralelização OpenMP, avaliando o desempenho da aplicação em ambiente de memória compartilhada.

Algorithm 1 Migração Reversa no Tempo (Mamute)

```
1: Receber o modelo de velocidades inicial m_0
2: Inicializar parâmetros e variáveis de simulação
3: pragma omp parallel {Início da região paralela OpenMP}
4: for cada tiro (shot) do
5:     {Leitura dos dados observados}
6:     Ler coordenadas dos receptores
7:     Ler traços sísmicos observados
8:     for cada passo de tempo t_i da propagação direta do
9:         Calcular campo de onda direto para o modelo m_0
10:    end for
11:    for cada passo de tempo t_i da retropropagação do
12:        Calcular campo de onda reverso
13:        Calcular a condição de imagem ponto a ponto
14:        Aplicar a compensação por iluminação
15:    end for
16: end for
17: {Fim da região paralela OpenMP}
```

Capítulo 4

PaScal Suite

O conjunto de ferramentas PaScal Suite tem como objetivo auxiliar os desenvolvedores na análise de escalabilidade de programas paralelos. Ele é composto pelas ferramentas PaScal Analyzer e PaScal Viewer.

4.1 PaScal Analyzer

O PaScal Analyzer é uma ferramenta projetada para avaliar a escalabilidade de aplicações, sendo compatível com programas desenvolvidos em C e C++. Ele permite a coleta automatizada de dados a partir de múltiplas execuções, o que facilita a comparação do desempenho entre diferentes configurações de paralelismo (da Silva et al. 2022).

Para realizar a coleta de dados no Analyzer, o usuário configura os parâmetros experimentais, como o número de núcleos de processamento, os tamanhos de entrada e o número de repetições. Além disso, a ferramenta permite delimitar e isolar regiões específicas do código por meio de uma instrumentação manual. Um exemplo de configuração via linha de comando é apresentado a seguir:

```
pascalanalyzer ./aplicacao -c 1,2,4,8,16 -i 25,50,100,200 -o saida.json
-r 4 -t aut
```

Em que:

- `-c`: determina a quantidade de núcleos de processamento, que, nesse caso, varia entre 1, 2, 4, 8 e 16;
- `-i`: define as entradas (tamanhos do problema) para a medição;
- `-o`: especifica o arquivo de saída (`saida.json`) gerado pelo PaScal Analyzer com os dados das medições;
- `-r`: indica a quantidade de repetições;
- `-t`: define o tipo da instrumentação utilizada na aplicação.

O fluxo de trabalho do PaScal Analyzer, conforme descrito por Gomes (2025) e ilustrado na Figura 4.1, inicia-se com a execução da ferramenta pelo usuário, utilizando os parâmetros experimentais previamente definidos (Passo 1). Em seguida, o PaScal Suite inicia a execução da aplicação com OpenMP (Passo 2). Para habilitar a coleta de métricas

durante essa execução, o código-fonte da aplicação a ser analisada precisa ser instrumentado, o que permite que cada *thread* do processo principal registre as suas medições de desempenho (Passo 3). Esse processo de instrumentação pode ocorrer de forma automática, por meio da busca e do encapsulamento de blocos paralelizados, ou de forma manual. Na abordagem manual, o usuário insere macros específicas no código-fonte (`pascal_start`, para marcar o início da região, e `pascal_stop`, para marcar o fim) com o objetivo de delimitar com precisão as regiões a serem analisadas. Durante a execução da aplicação, as medições de tempo de todas as *threads* ativas são enviadas via *sockets* UNIX de volta para a instância principal do PaScal Suite (Passo 4). Por fim, a ferramenta unifica esses dados, organiza-os em uma estrutura padronizada e gera o arquivo de saída no formato JSON (Passo 5).

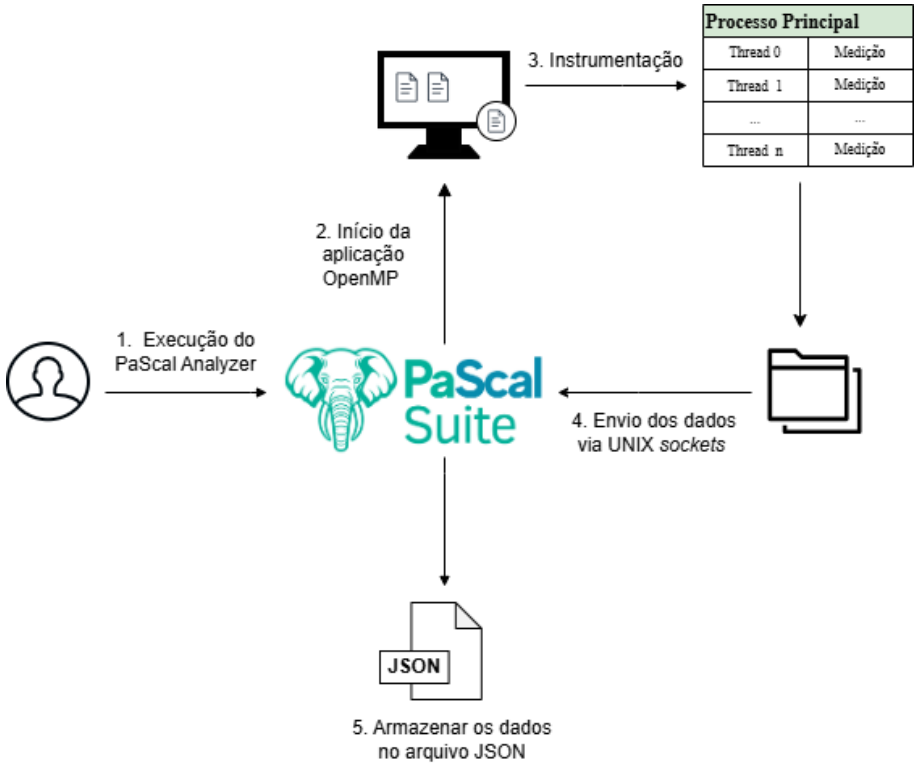


Figura 4.1: Fluxo de trabalho do PaScal Analyzer. Fonte: Adaptado de Gomes (2025).

Um exemplo da estrutura do arquivo JSON gerado pelo PaScal Analyzer, com alguns dados abreviados, pode ser observado a seguir. Nesse arquivo, o bloco `config` detalha o ambiente experimental: `execdate` indica a data da execução, a chave `command` mostra o comando de terminal executado pelo usuário, `hostname` identifica a máquina utilizada e `kernel` registra a versão do sistema operacional. Por sua vez, o bloco `data` armazena os resultados, mapeando a configuração do teste (como a quantidade de *threads* e a região analisada) aos respectivos tempos de início e fim da execução, os quais foram coletados no código-fonte por meio das macros `pascal_start` e `pascal_stop`.

```
{
```

```
  "config": {
    "pkg": "/aplicacao",
    "execdate": "08-03-2026_12:56:15",
    "command": "pascalanalyzer ./aplicacao -c 1,2,4,8,16
    -i 25,50,100,200 -o saida.json -t man",
    "hostname": "fat02.npad.internal",
    "kernel": "Linux-4.18.0-477.10.1.el8_8.x86_64-x86_64-with-glibc2.14",
    "disable_hyperthread": false,
    "auto_instrumentation": false,
    "idle_time": 0,
    "data_descriptor": {
      "values": ["start_time", "stop_time"],
      "extras": {
        "regions": {
          "values": [
            "start_time", "stop_time", "start_line",
            "stop_line", "thread_id", "filename"
          ]
        }
      },
      "keys": ["cores", "input", "repetitions"]
    },
    "disable_cores": false,
    "discard_ompnt": true,
    "arguments": ["25"]
  },
  "data": {
    "16;0;1": {
      "start_time": 1777218975.6963873,
      "stop_time": 1777218975.9006112
    },
    ...
  }
}
```


4.2 PaScal Viewer

O PaScal Viewer é uma aplicação *web* desenvolvida para facilitar a visualização de métricas de desempenho de programas paralelos (da Silva et al. 2019). A ferramenta pode ser acessada em: `https://pascalsuite.imd.ufrn.br/dev-ang-pascal-viewer/`. O PaScal Viewer recebe como entrada o arquivo no formato JSON gerado pelo PaScal Analyzer, o qual contém os tempos de execução da aplicação completa e de cada região instrumentada. A partir dos dados da execução sequencial e das execuções paralelas variando o número de núcleos, a aplicação realiza o cálculo das métricas de desempenho.

A ferramenta apresenta métricas essenciais, como *speedup* e eficiência, por meio de mapas de calor. Essa abordagem permite identificar se o programa é escalável e avaliar o seu comportamento sob as perspectivas de escalabilidade forte ou fraca, tanto de forma global quanto em regiões específicas do código-fonte.

A Figura 4.2 ilustra o modelo de mapa de calor gerado pelo PaScal Viewer a partir do arquivo JSON fornecido pelo Analyzer. Nessa representação, as cores mais quentes indicam eficiências mais altas (próximas a 100%), enquanto as cores mais frias evidenciam a queda de desempenho. O eixo vertical apresenta o número de núcleos de processamento utilizados, enquanto o eixo horizontal apresenta as entradas avaliadas, refletindo o tamanho do problema.

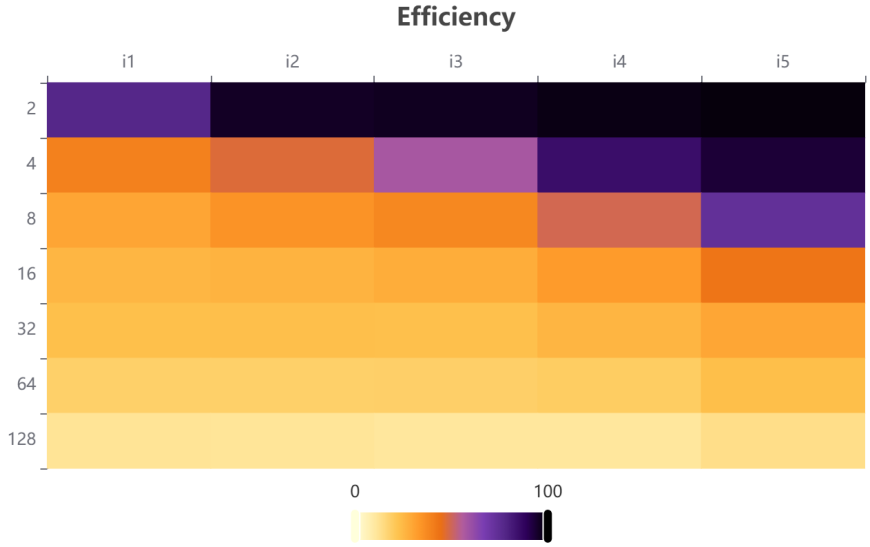


Figura 4.2: Diagrama de eficiência gerado pelo PaScal Viewer. O eixo vertical apresenta o número de núcleos variando entre 1, 2, 4, 8, 16, 32, 64 e 128. O eixo horizontal apresenta as entradas de i_1 a i_5.

Capítulo 5

Experimentos

Considerando que o objetivo deste trabalho é analisar a escalabilidade do algoritmo de RTM utilizando o conjunto de ferramentas PaScal Suite, o PaScal Analyzer foi utilizado para o perfilamento do código, avaliando o desempenho da aplicação por meio de configurações com 1, 2, 4, 8, 16, 32, 64, 128 e 256 *threads*. A métrica de execução sequencial adotada como referência consistiu na execução do próprio código paralelo utilizando apenas 1 *thread*. Entretanto, o uso de 1 *thread* embute o *overhead* inerente às diretivas e ao gerenciamento do OpenMP no tempo de referência. Uma comparação baseada no uso do código-fonte puramente sequencial demonstraria com maior exatidão a eficiência real da aplicação, visto que este tempo tende a ser menor do que o da versão paralela com uma única *thread*. No entanto, a adoção dessa estratégia foi necessária para viabilizar a automação dos experimentos por meio do PaScal Analyzer, o qual realiza as medições e gerencia as múltiplas configurações de recursos a partir de um único arquivo executável.

A execução foi realizada utilizando os cinco modelos de entrada, com cinco repetições para cada configuração. Além disso, foi aplicada a instrumentação manual para medir o tempo de execução em regiões específicas do algoritmo, como a propagação direta, que faz o cálculo do campo de onda direto; a retropropagação, que calcula o campo de onda reverso; e a condição de imagem, que correlaciona ponto a ponto ambos os campos. Estas regiões foram escolhidas por serem as etapas principais do RTM, com o objetivo de isolar e identificar possíveis gargalos de desempenho na aplicação.

O Algoritmo 2 detalha como a instrumentação manual foi estruturada no código-fonte, evidenciando as sub-regiões analisadas. Essa instrumentação foi realizada demarcando os trechos de interesse por meio da macro `pascal_start`, para indicar o início, e `pascal_stop`, para marcar o fim de cada bloco. Dessa forma, foi analisada a região da propagação direta em sua totalidade, bem como as sub-regiões internas responsáveis pelo cálculo do campo de onda direto, pelo salvamento dos *checkpoints* e pelo cálculo da iluminação para as fontes. Além disso, foi instrumentado o trecho do código correspondente à retropropagação, englobando as sub-regiões que realizam o cálculo do campo de onda reverso, a recuperação dos *checkpoints* (ou recomputação, caso não estejam salvos na memória), o cálculo da condição de imagem e a iluminação para os receptores.

Os experimentos foram realizados em um nó computacional pertencente ao supercomputador do Núcleo de Processamento de Alto Desempenho (NPAD)[1]. O equipamento

[1] https://npad.ufrn.br/npad/hardware

utilizado conta com dois processadores AMD EPYC 9654 com frequência base de 2,4 GHz (podendo atingir 3,7 GHz), totalizando 192 núcleos físicos de processamento e 384 threads, além de 3 TB de memória RAM.

Para as simulações, foram consideradas três dimensões espaciais (x_1, x_2 e x_3). Os parâmetros de modelagem numéricos foram mantidos constantes em todos os experimentos: frequência de pico da fonte de 30 Hz, amostragem temporal de 1 ms, resoluções espaciais de $\Delta x_1 = \Delta x_2 = \Delta x_3 = 10$m.

Foram utilizados cinco modelos de velocidades, denominados i_1, i_2, i_3, i_4 e i_5, descritos na Tabela 5.1. O modelo i_5 corresponde a um recorte do modelo de velocidades *Overthrust*, que é um modelo sintético amplamente utilizado na literatura devido à complexidade das falhas geológicas presentes em sua estrutura. Nesse cenário, as velocidades acústicas variam entre 2000 m/s e 6000 m/s. Os modelos i_1 a i_4 consistem em recortes do modelo i_5, restritos na dimensão x_2. A Figura 5.1 apresenta uma representação bidimensional (2D), nos eixos y e z, do modelo de velocidades tridimensional (3D) utilizado. Na ilustração, cada recorte realizado no volume está representado por um retângulo tracejado, sendo as cores branca, verde, amarela e magenta correspondentes aos modelos i_1 a i_4, respectivamente. Além disso, a condição de imagem utilizada foi a correlação cruzada, compensada pela iluminação da fonte e do receptor.

Tabela 5.1: Dimensões das malhas dos modelos de velocidades utilizados nos experimentos.

Modelo de Entrada	**Dimensões da Malha** (n_x, n_y, n_z)
i_1	$801, 51, 187$
i_2	$801, 101, 187$
i_3	$801, 201, 187$
i_4	$801, 401, 187$
i_5	$801, 801, 187$

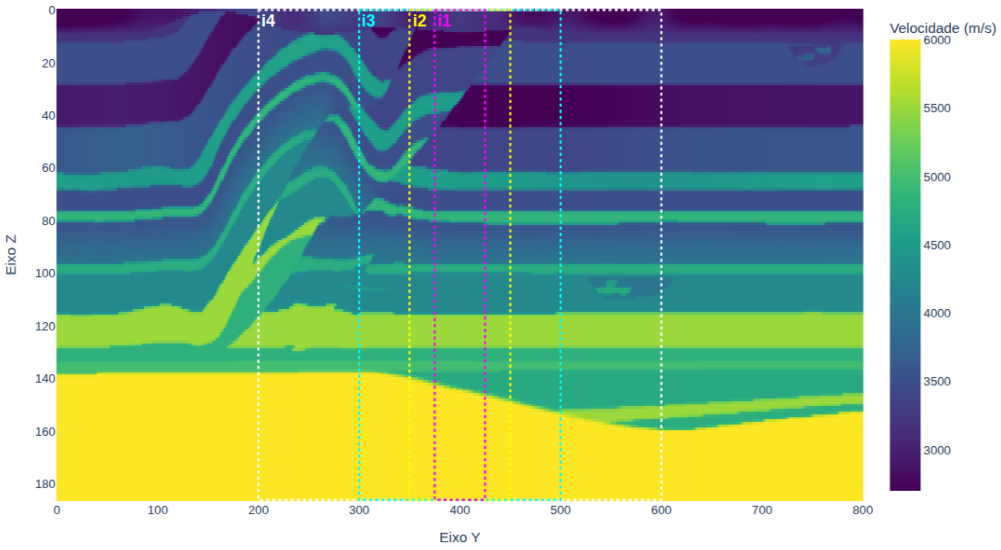


Figura 5.1: Representação bidimensional do modelo de velocidades com a delimitação dos recortes experimentais (i_1, i_2, i_3, i_4).

Algorithm 2 Migração Reversa no Tempo com Instrumentação Manual (PaScal Suite)

1: Receber o modelo de velocidades inicial m_0
2: Inicializar parâmetros e variáveis de simulação
3: **#pragma omp parallel** {Início da região paralela OpenMP}
4: **for** cada tiro (*shot*) **do**
5: {Leitura dos dados observados}
6: Ler coordenadas dos receptores
7: Ler traços sísmicos observados
8: **pascal_start(1)** {Início da Propagação Direta}
9: **for** $t_i = 0$ até n_s **do**
10: **pascal_start(2)**
11: **for** Todos os pontos no domínio do espaço **do**
12: Calcular campo de onda direto para o modelo m_0
13: **end for**
14: **pascal_stop(2)**
15: **if** t_i é um passo de salvamento (*checkpoint*) **then**
16: **pascal_start(3)**
17: Salvar *checkpoint* na memória
18: **pascal_stop(3)**
19: **end if**
20: **pascal_start(4)**
21: Calcular a iluminação para as fontes
22: **pascal_stop(4)**
23: **end for**
24: **pascal_stop(1)**
25: **pascal_start(5)** {Início da Retropropagação e Imageamento}
26: **for** $t_i = n_s$ até 0 **do**
27: **pascal_start(6)**
28: **for** Todos os pontos no domínio do espaço **do**
29: Calcular campo de onda reverso
30: **end for**
31: **pascal_stop(6)**
32: **pascal_start(7)**
33: Recomputar o campo de onda direto em t_i a partir dos *checkpoints*
34: **pascal_stop(7)**
35: **pascal_start(8)**
36: Calcular a condição de imagem ponto a ponto
37: **pascal_stop(8)**
38: **pascal_start(9)**
39: Calcular a iluminação para os receptores
40: **pascal_stop(9)**
41: **end for**
42: **pascal_stop(5)**
43: **end for**
44: {Fim da região paralela OpenMP}

Capítulo 6

Resultados

A Figura 6.1 apresenta a eficiência referente à execução completa da aplicação. Nela, é possível observar que o programa apresenta uma boa escalabilidade até 16 núcleos, mantendo a sua eficiência acima de 50%. Contudo, a partir desse ponto, o aumento do número de *threads* provoca uma degradação contínua da eficiência. Esse comportamento é esperado em aplicações limitadas pela largura de banda da memória (*memory-bound*), cenário em que o aumento de núcleos eleva a concorrência pelo acesso à memória compartilhada, além de incrementar o *overhead* de sincronização do OpenMP. Além disso, levando em consideração que a máquina utilizada conta com 192 núcleos físicos, é esperada uma queda de desempenho nas configurações que ultrapassam esse limite (como a execução com 256 *threads*), devido ao custo de sobreposição de contexto (*oversubscription*).

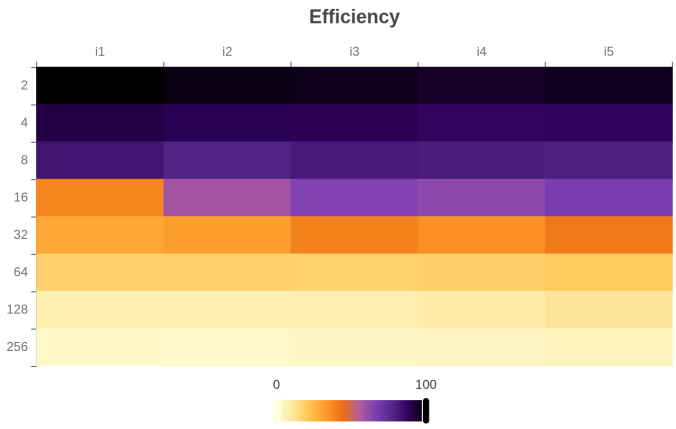
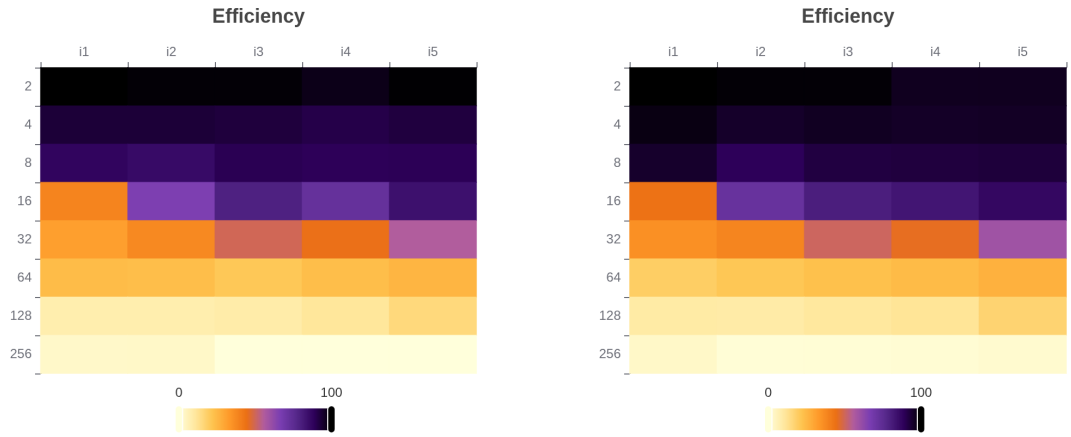


Figura 6.1: Diagrama de eficiência referente à execução completa da aplicação. O eixo vertical apresenta o número de núcleos variando entre 1, 2, 4, 8, 16, 32, 64, 128 e 256. O eixo horizontal reflete o tamanho do problema correspondente aos modelos i_1 a i_5. Os valores representam a mediana de cinco execuções.

Para uma investigação mais granular, a Figura 6.2 detalha a eficiência das regiões responsáveis pela propagação direta e retropropagação, respectivamente. Essas revelam que o comportamento global da aplicação é moldado por essas duas etapas principais, tendo em vista que a escalabilidade dessas regiões reflete o comportamento da execução

completa.



Propagação direta. Retropropagação.

Figura 6.2: Diagrama de eficiência comparativo das etapas de Propagação direta e Retropropagação. O eixo vertical apresenta o número de núcleos variando entre 1, 2, 4, 8, 16, 32, 64, 128 e 256. O eixo horizontal reflete o tamanho do problema correspondente aos modelos i_1 a i_5. Os valores representam a mediana de cinco execuções.

Entretanto, a retropropagação apresenta uma eficiência paralela ligeiramente superior em comparação à propagação direta. Isso pode ser atribuído ao fato de que a condição de imagem está inserida na etapa de retropropagação. Por exigir um alto volume de cálculos aritméticos iterativos em cada ponto da malha computacional, a condição de imagem possui uma alta densidade computacional, o que favorece a paralelização e ajuda a sustentar uma eficiência média mais alta nessa região específica do código, conforme ilustrado na Figura 6.3.

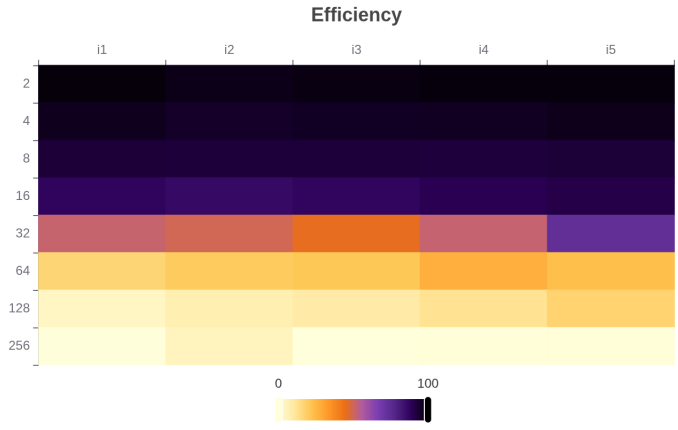
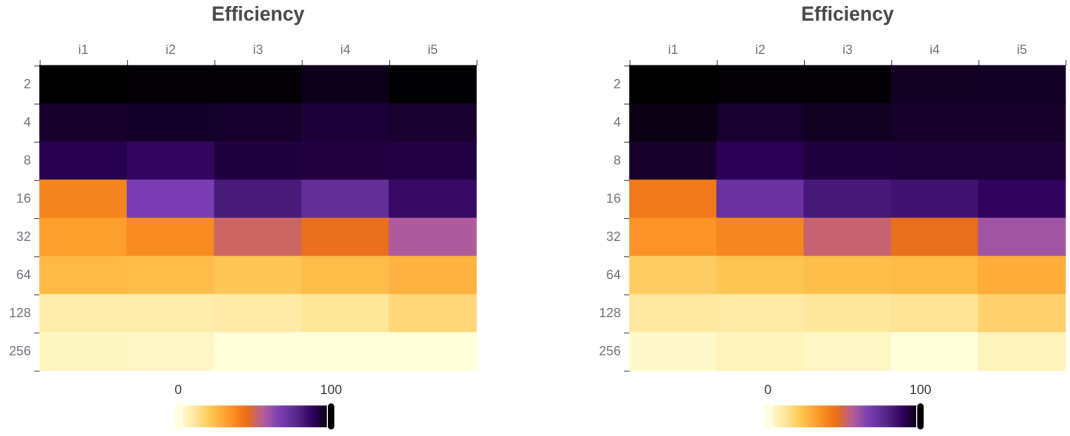


Figura 6.3: Diagrama de eficiência da Condição de Imagem. O eixo vertical apresenta o número de núcleos variando entre 1, 2, 4, 8, 16, 32, 64, 128 e 256. O eixo horizontal reflete o tamanho do problema correspondente aos modelos i_1 a i_5. Os valores representam a mediana de cinco execuções.

A Figura 6.4 apresenta a eficiência das sub-regiões de código responsáveis pelo cálculo do campo de onda direto e reverso, respectivamente. Nesta figura, é possível observar que a eficiência dessas regiões reflete o comportamento das propagações e, consequentemente, da execução completa da aplicação, evidenciando que a perda de desempenho está associada diretamente a essas etapas. Na etapa de propagação direta, o cálculo do campo de onda ocorre em conjunto com a inserção da fonte sísmica no modelo. Essa inserção, contudo, é realizada de forma sequencial por meio da diretiva OpenMP `#pragma omp single`, que garante que o bloco seja executado por apenas uma *thread*. Desse modo, essa restrição introduz uma fração sequencial ao algoritmo que justifica a queda mais acentuada da eficiência paralela observada nessa região.

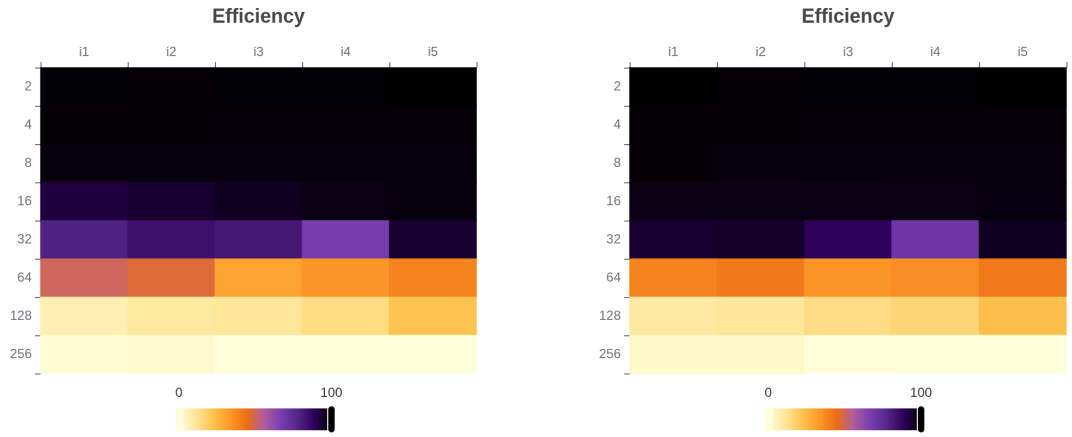
Em contrapartida, na etapa de retropropagação, a inserção dos dados observados é estruturada de forma concorrente, o que resulta em um melhor aproveitamento do paralelismo. No entanto, em ambas as etapas (propagação e retropropagação) é realizada a operação de *swap* (troca de ponteiros) para a atualização dos estados temporais do campo de onda. Esse processo de transição também é executado sequencialmente sob a restrição da diretiva `single`, o que impõe barreiras implícitas de sincronização, contribuindo para o *overhead* e limitando a escalabilidade da aplicação.



Cálculo do campo de onda direto. Cálculo do campo de onda reverso.

Figura 6.4: Diagrama de eficiência comparativo do cálculo de propagação direta e reversa. O eixo vertical apresenta o número de núcleos variando entre 1, 2, 4, 8, 16, 32, 64, 128 e 256. O eixo horizontal reflete o tamanho do problema correspondente aos modelos i_1 a i_5. Os valores representam a mediana de cinco execuções.

Além disso, nas etapas de propagação e retropropagação é aplicada a compensação por iluminação para as fontes e receptores, respectivamente. Nessas sub-regiões, os cálculos podem ser completamente paralelizados. A Figura 6.5 apresenta os diagramas de eficiência dessas operações, nas quais é possível observar uma escalabilidade consideravelmente superior ao restante da aplicação. Esse fenômeno ocorre porque a divisão de energia da iluminação consiste em cálculos independentes ponto a ponto na malha, sem dependência de dados entre as *threads*, permitindo o melhor aproveitamento do paralelismo.



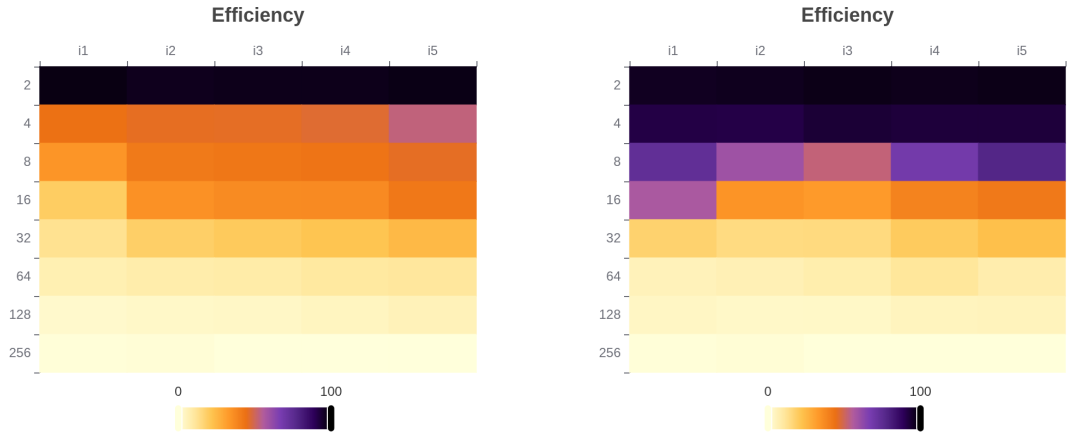
Iluminação da fonte. Iluminação do receptor.

Figura 6.5: Diagrama de eficiência comparativo das compensações por iluminação. O eixo vertical apresenta o número de núcleos variando entre 1, 2, 4, 8, 16, 32, 64, 128 e 256. O eixo horizontal reflete o tamanho do problema correspondente aos modelos i_1 a i_5. Os valores representam a mediana de cinco execuções.

A Figura 6.6 apresenta a eficiência das regiões do código responsáveis por salvar e recuperar os *checkpoints*. No salvamento dos *checkpoints*, a eficiência é de quase 100% com 2 *threads*. No entanto, ao aumentar os recursos computacionais, a eficiência cai para menos de 50%, com exceção do modelo i_5 com 4 núcleos, que atinge 52%. Esse comportamento pode ser atribuído ao fato de que, para salvar o *checkpoint*, é necessário acesso direto à memória ou ao disco para escrever os dados. Além disso, por ser uma região protegida pela diretiva `single` do OpenMP, a operação torna-se estritamente sequencial, o que gera ociosidade nas demais *threads* e degrada o desempenho paralelo.

Por outro lado, na recuperação dos *checkpoints*, se o dado estiver armazenado na memória, é feita uma cópia; se estiver no disco, o arquivo correspondente é lido. Para modelos pequenos (i_1), a eficiência nessa região mantém-se próxima de 100% até 16 *threads*. Contudo, o aumento contínuo de recursos degrada o desempenho na região. Para os modelos maiores, essa região de recuperação apresenta uma eficiência consistentemente menor que 50%.

Entretanto, é importante ressaltar que, embora algumas regiões apresentem uma eficiência ruim, como é o caso da operação de salvar os *checkpoints*, otimizá-las não garante, necessariamente, uma melhora significativa no desempenho geral da aplicação. Isso ocorre porque a colaboração dessa etapa no custo global é mínima, tendo em vista que o seu tempo de execução é pequeno quando comparado ao tempo da execução completa do programa.



Salvamento de *checkpoints* Recuperação de *checkpoints*

Figura 6.6: Diagrama de eficiência comparativo das regiões responsáveis por salvar e recuperar os *checkpoints*. O eixo vertical apresenta o número de núcleos variando entre 1, 2, 4, 8, 16, 32, 64, 128 e 256. O eixo horizontal reflete o tamanho do problema correspondente aos modelos i_1 a i_5. Os valores representam a mediana de cinco execuções.

Capítulo 7

Conclusão

A análise de escalabilidade do algoritmo de RTM implementado no software *Mamute*, utilizando a ferramenta PaScal Suite, foi essencial para o mapeamento das principais regiões que influenciam o desempenho total da aplicação, como a propagação e a retropropagação. Além disso, com os resultados foi possível observar que a eficiência global da aplicação se mantém acima de 50% até 16 *threads*, de modo que a partir desse limite, o aumento de recursos degrada o desempenho da aplicação devido à ociosidade das *threads*, revelando gargalos de paralelização e de acesso à memória compartilhada no código.

Também foi evidenciada a influência da diretiva `single` na eficiência de regiões específicas do código. Esse impacto foi observado na propagação (durante a inserção das fontes no modelo), na operação de *swap* para a troca de ponteiros do campo de onda (presente tanto na propagação quanto na retropropagação), e no processo de salvamento dos *checkpoints*.

Dessa forma, como trabalhos futuros, os próximos passos consistem em realizar otimizações focadas em mitigar os gargalos encontrados. Para minimizar a ociosidade das *threads* causada pela diretiva `single`, uma estratégia é avaliar a refatoração do código fazendo uso do modelo de tarefas (*tasking*) do OpenMP. Por fim, pretende-se comparar o novo desempenho alcançado com os resultados numéricos estabelecidos pela versão atual do algoritmo de RTM.

Referências Bibliográficas

Amdahl, Gene (1967), Validity of the single processor approach to achieving large scale computing capabilities, *em* 'Proceedings of the AFIPS spring joint computer conference', ACM, pp. 483–485.
URL: *http://dx.doi.org/10.1145/1465482.1465560*

Baysal, Edip, Dan D. Kosloff & John W. C. Sherwood (1983), 'Reverse time migration', *Geophysics* **48**(11), 1514–1524.

Carcione, José M., Gérard C. Herman & A. P. E. ten Kroode (2002), 'Seismic modeling', *Geophysics* **67**(4), 1304–1325.

Claerbout, Jon F. (1971), 'Toward a unified theory of reflector mapping', *Geophysics* **36**(3), 467–481.
URL: *http://geophysics.geoscienceworld.org/content/36/3/467.abstract*

Claerbout, Jon F. & Stephen M. Doherty (1972), 'Downward continuation of moveout-corrected seismograms', *Geophysics* .

da Silva, A. B., D. A. Cunha, V. R. Silva, A. F. de A. Furtunato & S. Xavier-de Souza (2019), PaScal viewer: A tool for the visualization of parallel scalability trends, *em* 'International Workshop on Extreme-Scale Programming Tools', Springer, pp. 250–264.

da Silva, Vitor Ramos Gomes, Anderson Bráulio Nóbrega da Silva, Carlos Valderrama, Pierre Manneback & Samuel Xavier-de Souza (2022), 'A minimally intrusive approach for automatic assessment of parallel performance scalability of sharedmemory HPC applications', *Electronics* **11**(5), 689.

Diaz, J., C. Muñoz-Caro & A. Niño (2012), 'A survey of parallel programming models and tools in the multi and many-core era', *IEEE Transactions on Parallel and Distributed Systems* **23**(8), 1369–1386.

Fernandes, João Batista (2020), Granularity load scheduling auto-tuning for multi-core processors applied to reverse-time migration, Mestrado, Universidade Federal do Rio Grande do Norte.

Gomes, J. F. P. (2025), Extending the PaScal analyzer for MPI scalability analysis: Design, implementation, and validation, B.s. thesis, Universidade Federal do Rio Grande do Norte.

Kosloff, Dan D. & Edip Baysal (1983), 'Migration with the full acoustic wave equation', *Geophysics* **48**(6), 677–687.

Lara, Matheus Alves Medeiros (2012), Paralelização de um algoritmo de propagação da onda acústica 2d usando MPI, Mestrado, Universidade Federal do Rio de Janeiro.

Lu, Ligang & Karen Magerlein (2013), 'Multi-level parallel computing of reverse time migration for seismic imaging on Blue Gene/Q', *SIGPLAN Not.* **48**(8), 291–292.

Mufti, Irshad R. (1990), 'Large-scale three-dimensional seismic models and their interpretive significance', *Geophysics* **55**(9), 1166–1182.

OpenMP Architecture Review Board (2015), *OpenMP Application Program Interface Version 4.5*.
URL: *https://www.openmp.org/wp-content/uploads/openmp-4.5.pdf*

Pacheco, Peter S. (2011), *An introduction to parallel programming*, Morgan Kaufmann, Amsterdam.

Silva, Max Wanlemberg Xavier (2009), Migração reversa no tempo com diferentes condições de imagem, Mestrado, COPPE - Universidade Federal do Rio de Janeiro.

Yilmaz, Öz (2001), *Seismic Data Analysis: Processing, Inversion, and Interpretation of Seismic Data*, Vol. 1 de *Investigations in geophysics*, Society of Exploration Geophysicists.

Zhou, Hua-Wei, Hao Hu, Zhihui Zou, Yukai Wo & Oong You (2018), 'Reverse time migration: A prospect of seismic imaging methodology', *Earth-Science Reviews* **179**, 207–227.